EDRIXS SciPy Backend: Implementation Note

August 11, 2026

Contents

1 Purpose and scope

This note documents the SciPy implementation of the staged EDRIXS solver interface. It is intended to be self-contained: a reader should be able to understand what the SciPy backend implements, how the main numerical objects are represented, and how the ED, XAS, and RIXS calculations are assembled without first reading the source code.

The SciPy backend is also the current reference implementation for other backends. A new backend such as PETSc does not need to reproduce the same numerical algorithms or storage formats, but it should reproduce the same operator meanings, public input/output conventions, and spectroscopy semantics.

The relevant source files are

```
edrixs/solvers.py
edrixs/scipy_backend/scipy_backend.py
edrixs/fock_basis.py
edrixs/krylov.py
edrixs/_solvers_helpers.py
```

The principal SciPy entry points are

```
build_op_scipy
ed_scipy
xas_scipy
rixs_scipy
```

with the numerical implementations

```
ed_krylov_scipy
xas_krylov_scipy
rixs_krylov_scipy
```

in the same backend module.

2 Where the SciPy backend fits in EDRIXS

The staged interface separates model definition from numerical representation and solution. A typical call chain is

```
models.py
    model_1v1c / model_2v1c / model_siam
        |
        v
solvers.py
    get_ops -> build_op
    ed
    xas
    rixs
        |
        v
scipy_backend/scipy_backend.py
    build_op_scipy
    ed_scipy
    xas_scipy
    rixs_scipy
```

The model layer produces backend-independent orbital-space information. For example, a model returns

```
emat_i, umat_i, basis_i,
emat_n, umat_n, basis_n,
trans_mat
```

where the suffixes `i` and `n` denote the initial/final and intermediate sectors, respectively. The SciPy backend is responsible for converting these objects into many-body operators and then solving the requested spectroscopy problem.

2.1 Backend dispatch

The public wrappers in `solvers.py` do not call SciPy-specific functions directly by name. They resolve a backend and then dispatch to the corresponding backend entry point. Thus

```
ed(..., backend="scipy")   -> ed_scipy(...)
xas(..., backend="scipy") -> xas_scipy(...)
rixs(..., backend="scipy") -> rixs_scipy(...)
```

The backend may also be inferred from already constructed operators. The SciPy recognizer

```
owns_operator_scipy(operator)
```

returns true for NumPy arrays, SciPy sparse matrices, and SciPy `LinearOperator` objects.

2.2 The role of `backend_kws`

Backend-specific numerical controls are deliberately grouped in one dictionary rather than exposed as backend-specific keywords in the public solver API. For example,

```
eval_i, evec_i = ed(
    hmat_i,
    num_evals=10,
    backend="scipy",
    backend_kws={
        "blocksize": 20,
        "tol": 1e-10,
        "maxiter": 1000,
    },
)
```

The public wrapper passes a copy of this dictionary to `ed_scipy`. The backend adapter then expands the dictionary into the arguments of `ed_krylov_scipy`. XAS and RIXS work in the same way.

This separation is important: physical arguments belong in the common interface; algorithm-specific controls belong in `backend_kws`.

3 Fock-basis representation

3.1 Integer encoding

Many-body basis states are represented by the shared `FockBasis` class in `fock_basis.py`. A basis state is encoded as an integer bit pattern. Orbital zero is stored as the most significant bit.

For a basis with N_{orb} spin-orbitals, the bit for orbital i is

$$b_i = 2^{N_{\mathrm{orb}}-1-i}. \tag{1}$$

A set bit means that the corresponding spin-orbital is occupied. `FockBasis.decode(j)` returns the integer state stored at many-body basis position j, while `FockBasis.encode(state)` returns the matrix index of an integer-encoded state.

The SciPy operator builder uses these operations directly. This basis convention is therefore part of the meaning of the many-body operators even though the `FockBasis` class itself is backend independent.

3.2 Fermionic signs

When applying creation or annihilation operators, the SciPy backend computes the usual fermionic parity from the number of occupied orbitals preceding the acted-on orbital. If $n_{<i}$ is the number of occupied orbitals before orbital i, the sign is

$$(-1)^{n_{<i}}. \tag{2}$$

The helper `_count_occupied_before` implements this count using bit operations on the integer-encoded Fock state.

4 Many-body operator construction

4.1 General contract

The public constructor is

```
build_op(emat, umat, lb, rb=None, backend="scipy", backend_kws=None)
```

which dispatches to

```
build_op_scipy(emat, umat, lb, rb=None, backend_kws=None)
```

for the SciPy backend.

This function constructs a *general many-body Fock-space operator*; it is not restricted to Hamiltonians. The operator maps states in the right basis `rb` to states in the left basis `lb`. If `rb` is omitted, it defaults to `lb`.

The resulting matrix has shape

$$(|\mathrm{lb}|, |\mathrm{rb}|). \tag{3}$$

This matters for photon transition operators, which generally map between different Hilbert spaces and are therefore not required to be square.

4.2 Two-fermion contribution

The matrix `emat` defines the two-fermion part

$$\hat{O}_2 = \sum_{ij} e_{ij}\, c_i^\dagger c_j. \tag{4}$$

The SciPy routine `two_fermion_csr` loops over non-negligible e_{ij} and over basis states in `right_basis`. It

1. checks that orbital j is occupied;
2. annihilates the electron at j, including the fermionic sign;
3. checks that orbital i is empty;
4. creates an electron at i, including the second fermionic sign;
5. looks up the resulting state in the left basis;
6. adds the resulting matrix element to a sparse COO accumulator.

The result is converted to CSR format.

4.3 Four-fermion contribution

For dense Coulomb data, the rank-four tensor is interpreted as

$$\hat{O}_4 = \sum_{lkji} U_{lkji}\, c_l^\dagger c_k^\dagger c_j c_i. \tag{5}$$

The SciPy implementation applies the operators from right to left: annihilate i, annihilate j, create k, create l, accumulating the fermionic sign at each operation.

The backend also accepts a flattened sparse representation of the four-index tensor. The flattening convention is

$$\text{row} = l\, N_{\text{orb}} + k, \tag{6}$$
$$\text{col} = j\, N_{\text{orb}} + i, \tag{7}$$

so that sparse matrix element (row, col) corresponds to U_{lkji}.

The helper `four_fermion_csr_auto` chooses automatically between dense and sparse input representations.

4.4 Assembly in `build_op_scipy`

`build_op_scipy` begins from a zero CSR matrix and adds the requested contributions independently:

```
operator = zero_matrix(shape=(len(lb), len(rb)))

if emat is not None:
    operator += two_fermion_csr(...)

if umat is not None:
    operator += four_fermion_csr_auto(...)
```

Therefore all of the following are valid:

`emat`	`umat`	Meaning
present	present	general interacting operator / Hamiltonian
present	`None`	two-fermion operator, including photon transitions
`None`	present	four-fermion-only operator
`None`	`None`	zero operator

The only current SciPy-specific construction option is

```
backend_kws={"tol": value}
```

with default 10^{-10}. Coefficients with magnitude not exceeding this threshold are ignored during construction. Unknown construction options raise an error.

4.5 Hamiltonians and photon transition operators

The public `get_ops` function uses `build_op` for all many-body operators.

The initial and intermediate Hamiltonians are constructed as

```
hmat_i = build_op(emat_i, umat_i, basis_i, ...)
hmat_n = build_op(emat_n, umat_n, basis_n, ...)
```

with the same basis on the left and right.

Each component of the photon transition operator is constructed as

```
build_op(
    component,
    None,
    basis_n,
    basis_i,
    backend="scipy",
)
```

and therefore has shape

$$(\dim \mathcal{H}_n, \dim \mathcal{H}_i). \tag{8}$$

There is no separate transition-operator construction API in the staged implementation.

5 Backend adapters

The public SciPy entry points

```
ed_scipy
xas_scipy
rixs_scipy
```

are deliberately thin. Their main jobs are to receive the common physical arguments, validate/-copy `backend_kws`, and call the corresponding numerical implementation.

Conceptually,

```
ed_scipy   -> ed_krylov_scipy
xas_scipy  -> xas_krylov_scipy
rixs_scipy -> rixs_krylov_scipy
```

This is useful when comparing SciPy with another backend: the adapter signatures define the backend contract, while the lower-level routines document one concrete numerical realization of that contract.

6 Ground-state diagonalization

6.1 Public and backend interfaces

The public call is

```
ed(hmat_i, num_evals=1, backend="scipy", backend_kws=None)
```

which dispatches to

```
ed_scipy(hmat_i, num_evals=1, backend_kws=None).
```

`ed_scipy` maps `num_evals` to the retained-state count used by `ed_krylov_scipy`.

6.2 LOBPCG implementation

The current numerical solver is SciPy's LOBPCG routine. The internal signature is

```
ed_krylov_scipy(
    hmat_i,
    num_gs=1,
    blocksize=None,
    *,
    tol=1e-10,
    maxiter=200,
    seed=None,
    initial_guess=None,
    suppress_lobpcg_warnings=True,
)
```

The Hamiltonian is converted to a SciPy `LinearOperator`; it must be square. If `blocksize` is not provided, it is set equal to the requested number of retained states. A block larger than the retained count is allowed. This is useful, for example, when low-energy degeneracies make a larger eigensolver block desirable.

If the caller does not provide `initial_guess`, the routine constructs a random real initial block using `numpy.random.default_rng(seed)`. If an initial guess is provided, it must have shape

$$(\dim \mathcal{H}_i, \text{blocksize}). \tag{9}$$

LOBPCG is called with `largest=False`. The returned eigenpairs are explicitly sorted by increasing eigenvalue. Only the first `num_gs` states are retained if a larger block was solved.

The output convention is

$$\texttt{eval\_i.shape} = (N_{\text{eval}},), \tag{10}$$

$$\texttt{evec\_i.shape} = (\dim \mathcal{H}_i, N_{\text{eval}}), \tag{11}$$

with eigenvectors stored as columns.

6.3 SciPy ED backend keywords

The current SciPy ED controls supplied through `backend_kws` are

Keyword	Default
`blocksize`	`None` (use retained-state count)
`tol`	10^{-10}
`maxiter`	200
`seed`	`None`
`initial_guess`	`None`
`suppress_lobpcg_warnings`	`True`

These are SciPy implementation controls, not common EDRIXS physical parameters.

7 Lanczos utilities used by spectroscopy

The shared `krylov.py` module provides the Lanczos tridiagonalization used by the SciPy XAS and final-state RIXS calculations.

Given a Hermitian operator H and a nonzero seed vector $|v_0\rangle$, `lanczos_tridiagonal` constructs the Krylov space

$$\mathcal{K}_m(H, v_0) = \mathrm{span}\{v_0, Hv_0, H^2v_0, \ldots\} \tag{12}$$

and produces the tridiagonal matrix

$$T = \begin{pmatrix} \alpha_0 & \beta_0 & & \\ \beta_0 & \alpha_1 & \beta_1 & \\ & \ddots & \ddots & \ddots \end{pmatrix}. \tag{13}$$

The routine returns

```
alpha, beta, norm
```

where `norm` is $\|v_0\|^2$, the squared norm of the original unnormalized seed.

The central projected-resolvent convention is

$$G(z) = \langle v_0|(zI - H)^{-1}|v_0\rangle, \tag{14}$$

approximated from the Lanczos tridiagonal representation as

$$G(z) \simeq \|v_0\|^2 \left[(zI - T)^{-1}\right]_{00}. \tag{15}$$

The corresponding broadened spectral function is

$$A(\omega) = -\frac{1}{\pi} \mathrm{Im}\, G(\omega + E_0 + i\eta). \tag{16}$$

The current Lanczos implementation uses the standard three-term recurrence and does not perform explicit reorthogonalization.

8 X-ray absorption spectroscopy (XAS)

8.1 Backend interface

The SciPy backend entry point is

```
xas_scipy(
    eval_i, evec_i, hmat_n, trans_op, ominc,
    gamma_c=0.1,
    thin=1.0,
    phi=0.0,
    pol_type=None,
    temperature=1.0,
    scatter_axis=None,
    backend_kws=None,
)
```

It forwards the physical arguments to `xas_krylov_scipy` and expands the SciPy-specific backend dictionary. The only current SciPy XAS numerical keyword is

$$\texttt{nkryl} = 200 \tag{17}$$

by default.

8.2 Input conventions

The retained initial states satisfy

$$\texttt{eval\_i.shape} = (N_i,), \tag{18}$$
$$\texttt{evec\_i.shape} = (\dim \mathcal{H}_i, N_i). \tag{19}$$

The intermediate Hamiltonian `hmat_n` must be square. Each photon transition operator must have shape

$$(\dim \mathcal{H}_n, \dim \mathcal{H}_i). \tag{20}$$

The number of transition components must be either 3 (dipole) or 5 (quadrupole).

8.3 Physical calculation

For one retained initial state $|i\rangle$, define a polarization-dependent transition operator

$$T_\epsilon = \sum_a \epsilon_a T_a, \tag{21}$$

where T_a are the three dipole or five quadrupole transition components and ϵ_a is the corresponding polarization vector in transition-operator space.

The starting vector is

$$|b_i\rangle = T_\epsilon |i\rangle. \tag{22}$$

The SciPy implementation performs a Lanczos tridiagonalization of H_n starting from $|b_i\rangle$. The resulting pole data are stored in the keys

```
eigval
npoles
norm
alpha
beta
```

for each retained initial state.

The final spectral function is generated by `get_spectra_from_poles`, which applies the core-hole broadening and thermal weights of the retained initial states.

8.4 Polarization handling

For linear, left-circular, or right-circular polarization, the code first constructs the physical dipole polarization vector. For a five-component quadrupole transition, this is combined with the photon wavevector to obtain the corresponding quadrupole polarization components.

For isotropic XAS, the SciPy implementation does not construct one coherent sum of all transition components. Instead, it evaluates the spectrum from each transition component separately and averages the resulting spectra over the number of components.

If `pol_type` is omitted, the SciPy XAS default is isotropic polarization.

8.5 Broadening and output

`gamma_c` may be either a scalar or a one-dimensional array of length `len(ominc)`. Scalars are expanded to the incident-energy grid.

The returned XAS array has shape

$$(\text{len}(\texttt{ominc}), \text{len}(\texttt{pol\_type})). \tag{23}$$

9 Resonant inelastic X-ray scattering (RIXS)

9.1 Backend interface

The SciPy backend entry point is

```
rixs_scipy(
    eval_i, evec_i,
    hmat_i, hmat_n,
    trans_op,
    ominc, eloss,
    gamma_c=0.1,
    gamma_f=0.01,
    thin=1.0,
    thout=1.0,
    phi=0.0,
    pol_type=None,
    temperature=1.0,
    scatter_axis=None,
    skip_gs=False,
    return_poles=False,
    backend_kws=None,
)
```

The SciPy numerical implementation is `rixs_krylov_scipy`.

9.2 Overall decomposition

The RIXS code is separated into three stages:

```
_prepare_rixs
_compute_rixs_records
_assemble_rixs
```

The calculation flow is

```
rixs_scipy
    |
    v
rixs_krylov_scipy
    |
    +-- _prepare_rixs
    |
    +-- _compute_rixs_records
    |       |
    |       +-- _rixs_krylov_one_contribution_scipy
    |
    +-- _assemble_rixs
```

This decomposition separates reusable problem setup, the independent physical contributions, and final spectral assembly.

9.3 Preparation

`_prepare_rixs` converts the Hamiltonians and transition operators to SciPy `LinearOperator` form and validates the dimensions.

The required operator shapes are

$$H_i : (\dim \mathcal{H}_i, \dim \mathcal{H}_i), \tag{24}$$

$$H_n : (\dim \mathcal{H}_n, \dim \mathcal{H}_n), \tag{25}$$

$$T_a : (\dim \mathcal{H}_n, \dim \mathcal{H}_i). \tag{26}$$

The transition operators must also provide an adjoint action, because RIXS needs $T_a^\dagger$.

The routine expands

- `gamma_c` over the incident-energy grid;
- `gamma_f` over the energy-loss grid;
- the incoming and outgoing polarization specifications into vectors in the 3- or 5-component transition-operator space.

If `skip_gs=True`, the retained initial-state eigenvectors are stored as the subspace that must later be excluded from the final-state response.

9.4 One independent contribution

One RIXS contribution is labeled by

$$(\omega_{\mathrm{in}},\, p,\, i), \tag{27}$$

where ω_{in} is one incident energy, p is one incoming/outgoing polarization pair, and i is one retained initial state.

For initial state $|i\rangle$, the incoming transition creates the intermediate-state right-hand side

$$|b_i\rangle = T_{\mathrm{in}}|i\rangle. \tag{28}$$

The correction vector $|x_i\rangle$ satisfies

$$\left[\omega_{\mathrm{in}} + E_i + i\Gamma_c - H_n\right]|x_i\rangle = |b_i\rangle. \tag{29}$$

The SciPy backend represents the shifted matrix as a `LinearOperator` and solves this linear system with GMRES.

The outgoing transition then produces a vector in the initial/final Hilbert space,

$$|f_i\rangle = T_{\mathrm{out}}^{\dagger}|x_i\rangle. \tag{30}$$

A final-state Lanczos calculation of H_i, seeded by $|f_i\rangle$, generates the poles used for the energy-loss spectrum.

9.5 GMRES implementation and controls

The current SciPy-specific RIXS controls are

Keyword	Default
`nkryl`	200
`linsys_tol`	10^{-9}
`linsys_maxiter`	50000
`linsys_restart`	200

They are passed through `backend_kws`.

The compatibility helper `_gmres_scipy_compat` accommodates old and new SciPy GMRES signatures. With a modern SciPy signature it calls

```
gmres(
    operator,
    rhs,
    rtol=tol,
    atol=0.0,
    restart=restart,
    maxiter=maxiter,
)
```

and with an older SciPy signature it uses `tol=tol`. A nonzero GMRES return code is treated as failure and raises `RuntimeError`.

9.6 Implementation of `skip_gs`

The meaning of `skip_gs` is that transitions into the retained initial-state subspace are excluded from the final-state RIXS spectrum.

In the SciPy implementation, after the outgoing transition has formed $|f_i\rangle$, the code projects out the subspace spanned by the retained columns of `evec_i`:

$$|f_i\rangle \leftarrow |f_i\rangle - V_i V_i^\dagger |f_i\rangle, \tag{31}$$

where

$$V_i = \begin{bmatrix} |i_1\rangle & |i_2\rangle & \cdots \end{bmatrix} \tag{32}$$

contains the retained initial-state eigenvectors.

In code this is

```
final_vector = final_vector - excluded_vectors @ (
    excluded_vectors.conj().T @ final_vector
)
```

when `excluded_vectors` is not `None`. The projection happens *before* the final-state Lanczos calculation, so the excluded subspace carries zero spectral weight.

This is a concrete SciPy implementation of the public `skip_gs` semantics. Another backend must reproduce the same physical operation using its own vector and matrix representation.

9.7 Serial contribution loop

`_compute_rixs_records` currently evaluates the contributions with ordinary nested loops over

```
incident energy
    polarization pair
        retained initial state
```

For each combination it constructs the incoming right-hand side and calls `_rixs_krylov_one_contribution_sc`

There are no public threading, process-pool, worker-count, or multiprocessing-start-method controls in the SciPy RIXS backend. Parallelism can instead be introduced above this level by making independent RIXS calls, for example with an incident-energy array of length one. SciPy itself may of course use lower-level native parallelism in linked numerical libraries.

9.8 Pole records and assembly

Each contribution returns a record containing

```
eigval
npoles
norm
alpha
beta
```

where `alpha`, `beta`, and `norm` are the final-state Lanczos data.

The records are stored with logical ordering

```
incident energy -> polarization -> retained initial state
```

and `_assemble_rixs` converts each group of retained-state records into an energy-loss spectrum using `get_spectra_from_poles`.

The final spectrum has shape

$$(\text{len}(\texttt{ominc}),\ \text{len}(\texttt{eloss}),\ \text{len}(\texttt{pol\_type})). \tag{33}$$

If `return_poles=False`, only this array is returned. If `return_poles=True`, the return value is

```
(spectrum, poles_all)
```

where `poles_all` is indexed first by incident energy and then by polarization.

10 Dense compatibility path

The name `dense` is currently a compatibility path rather than an independent dense numerical backend.

`build_op_dense` constructs the operator through the SciPy CSR implementation and converts the result to a NumPy array. The dense solver entry points are aliases:

```
ed_dense   = ed_scipy
xas_dense  = xas_scipy
rixs_dense = rixs_scipy
```

Thus the SciPy implementation currently services both explicit SciPy operators and the dense compatibility mode.

11 Behavior that defines the reference implementation

For another backend, the following are the important externally visible SciPy conventions to reproduce:

- `build_op` constructs a general many-body Fock-space operator, not only a Hamiltonian.
- The operator maps `rb` to `lb` and may be rectangular.
- Photon transition operators have shape $(\dim \mathcal{H}_n, \dim \mathcal{H}_i)$.
- Both 3-component dipole and 5-component quadrupole transitions are supported.
- ED returns lowest eigenvalues in ascending order with eigenvectors stored by column.
- XAS returns shape $(N_\omega, N_{\text{pol}})$.
- RIXS returns shape $(N_{\omega_{\text{in}}}, N_{\text{loss}}, N_{\text{pol}})$.
- Scalar or energy-dependent broadenings are accepted where supported by the public interface.
- Thermal weighting is applied consistently when spectra are assembled from retained initial states.
- `skip_gs=True` removes the retained initial-state subspace from the final-state RIXS response.
- `return_poles=True` returns the spectrum together with the nested pole data.

- Backend-specific numerical controls enter through a dictionary `backend_kws`.

The following are specifically SciPy implementation choices and need not be copied literally by another backend:

- CSR matrix storage;
- LOBPCG for ED;
- SciPy GMRES for the RIXS correction vector;
- the exact private helper decomposition;
- the SciPy-specific names and defaults inside `backend_kws`;
- NumPy/SciPy vector storage.

12 Compact algorithm summary

12.1 Operator construction

```
for each nonzero orbital-space coefficient:
    for each right-basis Fock state:
        apply annihilation/creation operators with fermionic signs
        if resulting state is in left basis:
            accumulate matrix element
return CSR operator
```

12.2 ED

```
convert H_i to LinearOperator
construct/validate initial LOBPCG block
solve for low eigenpairs
sort by eigenvalue
return requested lowest states
```

12.3 XAS

```
for each polarization:
    form polarization-weighted transition action
    for each retained initial state:
        start = T_in |i>
        Lanczos(H_n, start)
        store poles
    broaden and thermally weight poles on ominc grid
return spectrum
```

12.4 RIXS

```
prepare operators, broadenings, polarizations, skip_gs subspace
for each incident energy:
    for each polarization pair:
        for each retained initial state:
            rhs = T_in |i>
            solve (omega + E_i + i Gamma_c - H_n) x = rhs
            final = T_out^dagger x
            if skip_gs:
                project final out of retained initial-state subspace
            Lanczos(H_i, final)
            store pole record
assemble records into broadened energy-loss spectra
return spectrum, and optionally poles
```

13 Conclusion

The SciPy backend implements the complete staged EDRIXS workflow using a clear separation between backend-neutral physical inputs and SciPy-specific numerical machinery. Its operator builder translates orbital-space two- and four-fermion terms into sparse many-body operators; ED uses LOBPCG; XAS uses Lanczos pole expansions in the intermediate Hilbert space; and RIXS combines an intermediate correction-vector solve with a final-state Lanczos expansion. The public physical semantics are carried through the thin backend adapters, while method-specific controls remain isolated in `backend_kws`.

For future backends, the SciPy code should therefore be read in two ways: as an exact reference for the EDRIXS behavior and data conventions, and as only one possible realization of the underlying numerical algorithms.